

PONDICHERRY UNIVERSITY
RAMANUJAN SCHOOL OF MATHEMATICAL SCIENCES
DEPARTMENT OF STATISTICS



M.Sc. QUANTITATIVE FINANCE
SEMESTER III
LECTURE NOTES ON
Machine Learning in Quantitative Finance

PREPARED BY
NAME: N Rohit Vedhanandh
REG. NO.: 25MSQUFPY0002

OCTOBER 2026

Contents

1	Machine Learning Foundations & Data Preprocessing	1
1.1	Challenges in Machine Learning	1
1.2	Types of Variables	1
1.3	Data Preprocessing Pipeline	1
1.3.1	Scaling & Normalization	2
1.4	Train-Test-Validation Splitting	3
1.4.1	Common Partitioning Ratios	3
1.4.2	K-Fold Cross-Validation	3
1.4.3	Data Leakage Prevention	3
1.4.4	Splitting Financial Time-Series Data	3
1.5	Missing Data Mechanisms & Imputation	4
1.5.1	Missingness Mechanisms	4
1.5.2	Imputation Methods	4
1.5.3	Decision Framework for Selecting Imputation Methods	4
1.6	Imbalanced Categorical Data	5
1.6.1	Consequences of Class Imbalance	5
1.6.2	Remedial Solutions	5
1.7	Model Evaluation Metrics	5
1.8	Mistakes to be Avoided in ML Pipelines	6
1.9	Key Takeaways	6
2	K-Nearest Neighbors (KNN)	7
2.1	Introduction to KNN	7
2.1.1	Core Algorithmic Characteristics	7
2.1.2	Algorithmic Workflow	7
2.2	Distance Metrics	7
2.2.1	Euclidean Distance	7
2.2.2	Manhattan Distance	8
2.2.3	Minkowski Distance	8
2.3	Worked Numerical Examples	8
2.3.1	Fruit Classification via Euclidean Distance	8
2.3.2	Fintech Stock Recommendation with Standardized Indicators	10
2.3.3	Bank Loan Approval via Manhattan Distance	11
2.3.4	Real Estate House Price Prediction (KNN Regression)	12
2.4	Parameter Selection: Small k vs. Large k	13

Chapter 1

Machine Learning Foundations & Data Preprocessing

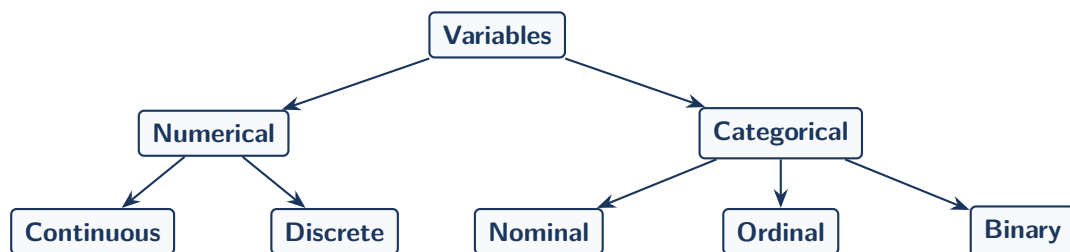
1.1 Challenges in Machine Learning

Machine learning applications in financial engineering face several structural and regulatory constraints:

- **Requires Large, Clean Data:** Models demand substantial volumes of high-quality, cleansed historical records to generalize effectively.
- **Risk of Overfitting:** Complex models risk fitting noise and idiosyncratic market movements rather than genuine underlying signals.
- **Interpretability (“Black Box”):** Advanced algorithms often lack transparent mechanistic explanations, creating auditing hurdles.
- **Regulatory & Ethical Concerns in Finance:** Strict regulatory compliance mandates model explainability, fairness, and accountability in financial decision-making.

1.2 Types of Variables

Variables encountered in machine learning datasets are broadly categorized into numerical and categorical types:

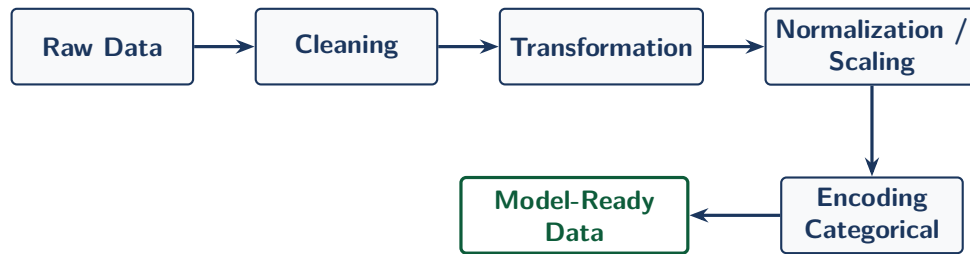


1.3 Data Preprocessing Pipeline

The standard end-to-end data preparation workflow transforms raw financial inputs into model-ready datasets:

Table 1.1: Taxonomy of Data Types and Financial Examples

Type	Subtype	Meaning	Finance Example
Numerical	Continuous	Any value in a continuous range	Stock price, portfolio return, volatility
Numerical	Discrete	Countable whole numbers	Number of trades per day, number of defaults
Categorical	Nominal	No natural ordering among categories	Sector: Banking, IT, Pharma
Categorical	Ordinal	Natural hierarchical order exists	Credit Rating: AAA > AA > A > BBB
Categorical	Binary	Exactly two distinct categories	Loan Default: Yes / No



- **Cleaning:** Removes duplicate records, corrects erroneous entries, and handles missing observations.
- **Transformation:** Converts raw figures into mathematically suitable modeling formats (e.g., calculating log returns from sequential stock prices).
- **Normalization / Scaling:** Brings numerical features across heterogeneous magnitudes into a common comparable scale.
- **Encoding:** Converts non-numeric categorical attributes into numerical indicator representations.

1.3.1 Scaling & Normalization

Features measured in disparate units must be rescaled to ensure distance-based and gradient-based algorithms treat features proportionately:

Min-Max Scaling

Rescales feature values into the bounded closed interval $[0, 1]$:

$$x' = \frac{x - x_{\min}}{x_{\max} - x_{\min}} \quad (1.1)$$

Z-Score Standardization

Standardizes observations using empirical sample mean and sample standard deviation:

$$Z = \frac{x - \mu}{\sigma} \quad (1.2)$$

1.4 Train-Test-Validation Splitting

Datasets are partitioned into distinct subsets to evaluate model performance on unseen data, verifying whether the algorithm generalizes rather than memorizing training samples:

- **Training Set:** Used to fit model parameters and learn patterns.
- **Validation Set:** Used during model development to tune hyperparameters and guide model selection.
- **Testing Set:** Used strictly for final unbiased model performance evaluation.

1.4.1 Common Partitioning Ratios

- **Train-Test Splits:** 80% – 20% or 70% – 30%.
- **Train-Validation-Test Splits:** 70% – 15% – 15%.

1.4.2 K-Fold Cross-Validation

In k -fold cross-validation:

1. The training dataset is partitioned into k equal parts (“folds”).
2. The model is trained on $k - 1$ folds and validated on the remaining fold.
3. This procedure is repeated k times, rotating the held-out validation fold sequentially across all splits.

Benefits & Common Choices

- Provides a significantly more robust estimate of true out-of-sample model performance.
- Reduces vulnerability to an anomalous single lucky or unlucky split.
- Common configurations: $k = 5$ or $k = 10$.

1.4.3 Data Leakage Prevention

Data Leakage Warning

Never scale or normalize the entire dataset prior to splitting.

If sample mean μ and standard deviation σ are calculated over the aggregate dataset, information from the test set leaks directly into the training stage.

Mandatory Pipeline Sequence:

1. Split the raw dataset first.
2. Fit the scaler/imputer exclusively on training data.
3. Transform validation and test data using the saved training parameters.

1.4.4 Splitting Financial Time-Series Data

For time-series data like stock prices, a purely random split is fundamentally problematic because it mixes past and future observations. Sequential financial records must strictly use **chronological splits**.

1.5 Missing Data Mechanisms & Imputation

1.5.1 Missingness Mechanisms

Table 1.2: Mechanisms of Missing Data and Practical Financial Examples

Type	Meaning	Finance Example
MCAR	Missing Completely at Random: Missingness is entirely independent of observed and unobserved variables.	Trade entry terminal intermittently experiences random network packet drops logging trades.
MAR	Missing at Random: Missingness is statistically related to other observed features in the dataset.	Younger retail banking customers are systematically more likely to leave the optional income field blank.
MNAR	Missing Not at Random: Missingness is directly correlated with the actual missing value itself.	Highly indebted distressed borrowers deliberately omit outstanding secondary credit obligations.

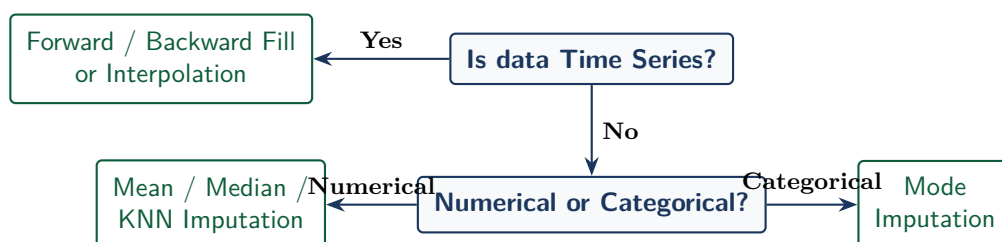
1.5.2 Imputation Methods

Common techniques for replacing missing data include: • Mean / Median / Mode, • K-Nearest Neighbors (KNN) Imputation, • Forward Fill, • Backward Fill, and • Interpolation.

Table 1.3: Advantages and Disadvantages of Imputation Methods

Method	Advantages	Disadvantages
Mean / Median	Simple, fast to compute	Distorts sample variance; ignores inter-feature relationships.
Mode	Directly applicable to categorical data	Can artificially over-represent the most common category.
KNN	Exploits multivariate relationships between features	Computationally expensive on large financial datasets.
Forward / Backward Fill	Well-suited for time-series sequences	Assumes most recent observed value remains representative.
Interpolation	Produces smooth estimates capturing trends	Unsuitable for non-sequential, unordered data.

1.5.3 Decision Framework for Selecting Imputation Methods



1.6 Imbalanced Categorical Data

Class imbalance arises when one category of the target variable appears far more frequently than other categories:

Financial Example

In credit card fraud detection:

- Genuine Transactions: 99.9%
- Fraudulent Transactions: 0.1%

A naive model that blindly predicts “no fraud” every single time achieves 99.9% raw accuracy, yet is entirely useless. Thus, **accuracy alone is fundamentally misleading**.

1.6.1 Consequences of Class Imbalance

- Biases model parameters heavily toward majority class patterns.
- Renders overall accuracy metric uninformative for decision-makers.

1.6.2 Remedial Solutions

- **Oversampling**: Duplicating or synthetically generating minority class instances.
- **Undersampling**: Down-sampling majority class observations.
- **SMOTE**: Synthetic Minority Over-sampling Technique, interpolating new instances along minority feature lines.

1.7 Model Evaluation Metrics

For binary classification problems, predictions and actual outcomes are summarized in a confusion matrix:

Table 1.4: Binary Classification Confusion Matrix

	Predicted Positive	Predicted Negative
Actual Positive	True Positive (TP)	False Negative (FN)
Actual Negative	False Positive (FP)	True Negative (TN)

Precision

Of all instances predicted as positive, how many were genuinely correct?

$$\text{Precision} = \frac{TP}{TP + FP} \quad (1.3)$$

Recall

Of all actual positive instances in the population, how many did the model identify?

$$\text{Recall} = \frac{TP}{TP + FN} \quad (1.4)$$

F_1 -Score

Harmonic mean of Precision and Recall, balancing false alarms against missed detections:

$$F_1 = 2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}} \quad (1.5)$$

1.8 Mistakes to be Avoided in ML Pipelines

1. **Global Scaling Before Splitting:** Scaling or normalizing the entire dataset prior to train/test partitioning causes data leakage.
2. **Blind Mean Imputation on MNAR Data:** Imputing missing values with unconditional means when data is Missing Not at Random injects substantial estimation bias.
3. **Ignoring Class Imbalance:** Relying exclusively on overall classification accuracy in skewed datasets conceals poor minority detection.
4. **Unchecked One-Hot Encoding on High-Cardinality Variables:** Creating dummy variables for high-cardinality categorical features leads to severe dimensionality inflation.
5. **Failing to Shuffle Non-Time-Series Data:** Neglecting random shuffling before partitioning standard cross-sectional data risks clustering-induced bias.

1.9 Key Takeaways

Core Principles of ML Data Preparation

- i) **Pre-processing dominates:** Data preprocessing accounts for 60% – 80% of total machine learning workflow effort.
- ii) **Always split first:** Partition data into training, validation, and test sets before performing any scaling or modeling.
- iii) **Diagnose missingness:** Determine whether missing values are MCAR, MAR, or MNAR before selecting an imputation strategy.
- iv) **Address class imbalance:** Use SMOTE, oversampling, or undersampling; accuracy alone is never sufficient.
- v) **Respect temporal order:** For financial and time-series data, strictly employ chronological splits and forward-looking imputation to prevent look-ahead bias.

Chapter 2

K-Nearest Neighbors (KNN)

2.1 Introduction to KNN

Definition 2.1.1: K-Nearest Neighbors

K-Nearest Neighbors (KNN) is a non-parametric, distance-based supervised machine learning algorithm used for both classification and regression tasks.

2.1.1 Core Algorithmic Characteristics

- **Supervised Learning:** Operates on labeled training datasets to predict continuous targets or discrete classes.
- **Dual Capability:** Utilized for classification (via majority voting) as well as regression (via local averaging).
- **Distance-Based Metric Method:** Assigns predictions purely based on geometric proximity in the feature space.
- **Lazy Learning / Instance-Based:** Does not construct an explicit internal model or parameter set during training; retains the entire training dataset and defers computation until inference time.

2.1.2 Algorithmic Workflow

The operational execution pipeline of KNN proceeds through four sequential stages:



2.2 Distance Metrics

Geometric closeness between query observations and stored instances is computed using formal metric distance functions.

2.2.1 Euclidean Distance

For two observations in a 2-dimensional feature space, $\mathbf{X} = (x_1, x_2)$ and $\mathbf{Y} = (y_1, y_2)$:

$$d(\mathbf{X}, \mathbf{Y}) = \sqrt{(x_1 - y_1)^2 + (x_2 - y_2)^2} \quad (2.1)$$

For an arbitrary p -dimensional feature vector (or n features):

$$d(\mathbf{X}, \mathbf{Y}) = \sqrt{\sum_{j=1}^p (x_j - y_j)^2} = \sqrt{\sum_{i=1}^n (x_i - y_i)^2} \quad (2.2)$$

2.2.2 Manhattan Distance

The Manhattan distance (L_1 norm) measures rectilinear city-block distance across coordinates:

$$d(\mathbf{X}, \mathbf{Y}) = \sum_{i=1}^n |x_i - y_i| \quad (2.3)$$

- **For 2 Dimensions:** $d = |x_1 - x_2| + |y_1 - y_2|$
- **For 3 Dimensions:** $d = |x_1 - x_2| + |y_1 - y_2| + |z_1 - z_2|$

2.2.3 Minkowski Distance

The generalized Minkowski metric of order $p \geq 1$ encompasses both Manhattan and Euclidean distances:

$$d(\mathbf{X}, \mathbf{Y}) = \left(\sum_{i=1}^n |x_i - y_i|^p \right)^{1/p} \quad (2.4)$$

Table 2.1: Minkowski Distance Orders and Resulting Metrics

Order p	Distance Metric	Mathematical Formula
$p = 1$	Manhattan distance	$\sum x_i - y_i $
$p = 2$	Euclidean distance	$\sqrt{\sum (x_i - y_i)^2}$
$p \geq 1$	Minkowski distance	$\left(\sum x_i - y_i ^p \right)^{1/p}$

Where:

- n = number of dimensions (features)
- p = positive real number (the “order” of the distance metric)
- x_i, y_i = the i -th feature coordinates of the two compared observations

2.3 Worked Numerical Examples

2.3.1 Fruit Classification via Euclidean Distance

Example 2.3.1: Fruit Classification (Apple vs. Lemon)

A dataset contains observations evaluated on two physical attributes: *Sweetness* and *Crunchiness*.

Fruit Training Dataset

Point	Sweetness	Crunchiness	Class
<i>A</i>	7	7	Apple
<i>B</i>	8	6	Apple
<i>C</i>	3	8	Lemon
<i>D</i>	2	7	Lemon
<i>E</i>	6	8	Apple
<i>F</i>	1	9	Lemon

Task: Classify a new query observation $\mathbf{X} = (5, 7)$ using $k = 3$ and Euclidean distance.

Solution:

Step 1: Compute Euclidean distances from query point $\mathbf{X}(5, 7)$:

$$\text{To } A(7, 7) : d = \sqrt{(5 - 7)^2 + (7 - 7)^2} = \sqrt{4 + 0} = 2.00$$

$$\text{To } B(8, 6) : d = \sqrt{(5 - 8)^2 + (7 - 6)^2} = \sqrt{9 + 1} = \sqrt{10} \approx 3.16$$

$$\text{To } C(3, 8) : d = \sqrt{(5 - 3)^2 + (7 - 8)^2} = \sqrt{4 + 1} = \sqrt{5} \approx 2.23$$

$$\text{To } D(2, 7) : d = \sqrt{(5 - 2)^2 + (7 - 7)^2} = \sqrt{9 + 0} = 3.00$$

$$\text{To } E(6, 8) : d = \sqrt{(5 - 6)^2 + (7 - 8)^2} = \sqrt{1 + 1} = \sqrt{2} \approx 1.41$$

$$\text{To } F(1, 9) : d = \sqrt{(5 - 1)^2 + (7 - 9)^2} = \sqrt{16 + 4} = \sqrt{20} \approx 4.04$$

Step 2: Sort distances in ascending order:

Point	Distance	Class
<i>E</i>	1.41	Apple
<i>A</i>	2.00	Apple
<i>C</i>	2.23	Lemon
<i>D</i>	3.00	Lemon
<i>B</i>	3.16	Apple
<i>F</i>	4.04	Lemon

Step 3: Evaluate majority class for $k = 3$:

$$\{k = 3\} = \{E, A, C\}$$

- Apple: 2 votes (E, A)
- Lemon: 1 vote (C)

$\therefore \mathbf{X}$ is to be classified as **Apple**.

2.3.2 Fintech Stock Recommendation with Standardized Indicators

Example 2.3.2: Fintech Stock Recommendation (Buy, Hold, Avoid)

A fintech company builds a KNN system to recommend whether a stock is a **Buy**, **Hold**, or **Avoid** based on three financial indicators: Price-to-Earnings (P/E), Return on Equity (ROE), and Debt-to-Equity (D/E).

The training samples are provided after standardizing each indicator via $Z = \frac{x-\mu}{\sigma}$:

Standardized Financial Indicator Training Dataset

Stock	$P/E (Z)$	$ROE (Z)$	$D/E (Z)$	Class
S_1	-1.351	1.542	-1.184	Buy
S_2	-0.787	0.806	-0.858	Buy
S_3	-0.129	-0.299	-0.204	Hold
S_4	0.341	-0.668	0.286	Hold
S_5	1.750	-1.404	1.757	Avoid
S_6	1.093	-1.036	1.267	Avoid
S_7	-1.069	1.174	-1.021	Buy
S_8	0.153	-0.115	-0.041	Hold

Task: Predict the recommendation class for a new stock with standardized features:

$$\mathbf{X} = (-0.559, 0.437, -0.531)$$

using $k = 3$ and Euclidean distance.

Solution:

Step 1: Compute Euclidean distance for each stock:

$$d(\mathbf{X}, S_i) = \sqrt{(z_{1,X} - z_{1,S_i})^2 + (z_{2,X} - z_{2,S_i})^2 + (z_{3,X} - z_{3,S_i})^2}$$

Stock	$P/E (Z)$	$ROE (Z)$	$D/E (Z)$	Distance	Class
S_1	-1.351	1.542	-1.184	1.487	Buy
S_2	-0.787	0.806	-0.858	0.527	Buy
S_3	-0.129	-0.299	-0.204	0.939	Hold
S_4	0.341	-0.668	0.286	1.266	Hold
S_5	1.750	-1.404	1.757	3.760	Avoid
S_6	1.093	-1.036	1.267	2.874	Avoid
S_7	-1.069	1.174	-1.021	1.002	Buy
S_8	0.153	-0.115	-0.041	1.053	Hold

Step 2: Sorted Euclidean distance table:

Stock	Distance	Class
S_2	0.527	Buy
S_3	0.939	Hold
S_7	1.002	Buy
S_8	1.053	Hold
S_4	1.266	Hold
S_1	1.487	Buy
S_6	2.874	Avoid
S_5	3.760	Avoid

Step 3: Majority voting for $k = 3$:

$$\{k = 3\} = \{S_2, S_3, S_7\}$$

- Buy: 2 votes (S_2, S_7)
- Hold: 1 vote (S_3)
- Avoid: 0 votes

$\therefore$ The new point is to be classified as **Buy** for $k = 3$.

2.3.3 Bank Loan Approval via Manhattan Distance

Example 2.3.3: Bank Loan Approval Decision

A bank utilizes KNN to decide whether to **Approve** or **Reject** a loan application based on two features:

- Credit score (scaled by dividing by 100, such that $750 \implies 7.5$)
- Annual Income (in Rs. Lakhs)

Loan Applicant Training Dataset

Applicant	Credit Score	Annual Income (Rs. Lakhs)	Decision
A_1	7.5	8.0	Approve
A_2	6.8	6.0	Approve
A_3	6.0	4.0	Reject
A_4	5.5	3.0	Reject
A_5	7.2	7.0	Approve
A_6	5.8	3.5	Reject

Task: Predict the decision for a new applicant with Credit Score = 6.5 and Annual Income = 5.5 Lakhs, i.e., $\mathbf{X} = (6.5, 5.5)$, using $k = 3$ and Manhattan distance.

Solution:

Step 1: Compute Manhattan distances from query point $\mathbf{X}(6.5, 5.5)$:

$$\text{To } A_1(7.5, 8.0) : d = |6.5 - 7.5| + |5.5 - 8.0| = 1.0 + 2.5 = 3.5$$

$$\text{To } A_2(6.8, 6.0) : d = |6.5 - 6.8| + |5.5 - 6.0| = 0.3 + 0.5 = 0.8$$

$$\text{To } A_3(6.0, 4.0) : d = |6.5 - 6.0| + |5.5 - 4.0| = 0.5 + 1.5 = 2.0$$

$$\text{To } A_4(5.5, 3.0) : d = |6.5 - 5.5| + |5.5 - 3.0| = 1.0 + 2.5 = 3.5$$

$$\text{To } A_5(7.2, 7.0) : d = |6.5 - 7.2| + |5.5 - 7.0| = 0.7 + 1.5 = 2.2$$

$$\text{To } A_6(5.8, 3.5) : d = |6.5 - 5.8| + |5.5 - 3.5| = 0.7 + 2.0 = 2.7$$

Step 2: Sorted Manhattan distance ranking:

Applicant	Distance	Decision
A_2	0.8	Approve
A_3	2.0	Reject
A_5	2.2	Approve
A_6	2.7	Reject
A_4	3.5	Reject
A_1	3.5	Approve

Step 3: Majority vote for $k = 3$:

$$\{k = 3\} = \{A_2, A_3, A_5\}$$

- Approve: 2 votes (A_2, A_5)
- Reject: 1 vote (A_3)

$\therefore$ The new applicant $\mathbf{X} = (6.5, 5.5)$ is to be concluded as **Approve** when $k = 3$.

2.3.4 Real Estate House Price Prediction (KNN Regression)**Example 2.3.4: Real Estate Price Estimation (KNN Regression)**

A real estate agency uses KNN regression to estimate house prices based on:

- Area (in hundreds of sq. ft., e.g., 1400 sq. ft. $\implies$ 14)
- Age (in years)

Real Estate Housing Dataset

House	Area (hundred sq. ft.)	Age (years)	Price (Rs. Lakhs)
H_1	12	5	50
H_2	15	3	65
H_3	8	10	30
H_4	20	2	90
H_5	10	8	38
H_6	18	6	75

Task: Predict the price of a new house with Area = 14 (1400 sq. ft.) and Age = 4 years, i.e., $\mathbf{X} = (14, 4)$, using $k = 3$ and Manhattan distance.

Solution:

Step 1: Compute Manhattan distances from query house $\mathbf{X}(14, 4)$:

$$\text{To } H_1(12, 5) : d = |14 - 12| + |4 - 5| = 2 + 1 = 3$$

$$\text{To } H_2(15, 3) : d = |14 - 15| + |4 - 3| = 1 + 1 = 2$$

$$\text{To } H_3(8, 10) : d = |14 - 8| + |4 - 10| = 6 + 6 = 12$$

$$\text{To } H_4(20, 2) : d = |14 - 20| + |4 - 2| = 6 + 2 = 8$$

$$\text{To } H_5(10, 8) : d = |14 - 10| + |4 - 8| = 4 + 4 = 8$$

$$\text{To } H_6(18, 6) : d = |14 - 18| + |4 - 6| = 4 + 2 = 6$$

Step 2: Sorted Manhattan distance ranking:

House	Distance	Price (Rs. Lakhs)
H_2	2	65
H_1	3	50
H_6	6	75
H_4	8	90
H_5	8	38
H_3	12	30

Step 3: Average prices across $k = 3$ nearest neighbors:

$$\{k = 3\} = \{H_2, H_1, H_6\}$$

$$\hat{y} = \frac{65 + 50 + 75}{3} = \frac{190}{3} \approx 63.33 \text{ Lakhs}$$

$\therefore$ The new point $\mathbf{X} = (14, 4)$ is predicted to have a price of **Rs. 63.33 Lakhs** for $k = 3$.

2.4 Parameter Selection: Small k vs. Large k

Selecting the hyperparameter k balances local sensitivity against global regularization:

Table 2.2: Comparison of Small k versus Large k Characteristics

Aspect	Small k	Large k
Neighbourhood	Local	Broad
Sensitivity to Noise	High	Lower
Variance	Higher	Lower
Bias	Lower	Higher
Decision Boundary	Irregular / complex	Smoother
Risk	Overfitting	Underfitting

Guiding Principle for Choosing k

Choose k to balance bias and variance.

Chapter 3

Naive Bayes Classification

3.1 Probability Foundations & Notations

The Naive Bayes classifier is grounded in classical axiomatic probability theory and conditional distributions.

Table 3.1: Probability Notations and Terminology

Notation	Meaning
$P(A)$	Probability of event A
$P(A^c)$	Probability of complement of A
$P(A \cap B)$	Joint probability of A and B occurring simultaneously
$P(A \cup B)$	Probability of either event A or event B occurring
$P(A B)$	Conditional probability of event A given event B has occurred
$P(A, B)$	Joint probability of events A and B (alternative standard notation)

3.1.1 Key Probability Identities

- **Complement Rule:**

$$P(A^c) = 1 - P(A) \quad (3.1)$$

- **Addition Rule (Union):**

$$P(A \cup B) = P(A) + P(B) - P(A \cap B) \quad (3.2)$$

- **Joint Probability Equivalence:**

$$P(A \cap B) = P(A, B) \quad (3.3)$$

3.1.2 Conditional Probability & Multiplication Rule

Definition 3.1.1: Conditional Probability

For two events A and B with $P(B) > 0$:

$$P(A | B) = \frac{P(A \cap B)}{P(B)} \quad (3.4)$$

Re-arranging yields the general **Multiplication Rule**:

$$P(A, B) = P(A | B)P(B) \quad (3.5)$$

$$P(A, B) = P(B | A)P(A) \quad (3.6)$$

3.1.3 Statistical Independence

If events A and B are statistically independent:

$$P(A \cap B) = P(A) \cdot P(B) \quad (3.7)$$

and equivalently:

$$P(A | B) = P(A) \quad (3.8)$$

3.2 Bayes' Theorem

Suppose an analyst wishes to determine the probability that an observation belongs to class C_k after observing feature evidence vector $\mathbf{X}$:

Theorem 3.2.1: Bayes' Theorem

$$P(C_k | \mathbf{X}) = \frac{P(\mathbf{X} | C_k)P(C_k)}{P(\mathbf{X})} \quad (3.9)$$

In conceptual terms:

$$\text{Posterior} = \frac{\text{Likelihood} \times \text{Prior}}{\text{Evidence}} \quad (3.10)$$

3.3 Worked Numerical Example: Credit Risk Classification

Example 3.3.1: Credit Risk Classification

A retail lending institution assesses credit risk with two target states:

- C_1 : Good Credit
- C_2 : Default

Historical portfolio priors and likelihoods for a financial indicator vector $\mathbf{X}$ are given by:

$$\begin{aligned} P(C_1) &= 0.80, & P(C_2) &= 0.20 \\ P(\mathbf{X} | C_1) &= 0.10, & P(\mathbf{X} | C_2) &= 0.60 \end{aligned}$$

Task: Compute the evidence $P(\mathbf{X})$, determine the posterior probabilities $P(C_1 | \mathbf{X})$ and $P(C_2 | \mathbf{X})$, and classify the borrower.

Solution:

Step 1: Compute Total Evidence $P(\mathbf{X})$ via Law of Total Probability:

$$\begin{aligned} P(\mathbf{X}) &= P(\mathbf{X} | C_1)P(C_1) + P(\mathbf{X} | C_2)P(C_2) \\ &= (0.10 \times 0.80) + (0.60 \times 0.20) \\ &= 0.08 + 0.12 \\ &= 0.20 \end{aligned}$$

Step 2: Calculate Posterior Probabilities:

$$P(C_1 | \mathbf{X}) = \frac{P(\mathbf{X} | C_1)P(C_1)}{P(\mathbf{X})} = \frac{0.10 \times 0.80}{0.20} = \frac{0.08}{0.20} = 0.40$$

$$P(C_2 | \mathbf{X}) = \frac{P(\mathbf{X} | C_2)P(C_2)}{P(\mathbf{X})} = \frac{0.60 \times 0.20}{0.20} = \frac{0.12}{0.20} = 0.60$$

Step 3: Comparison and Decision:

$$P(C_2 | \mathbf{X}) > P(C_1 | \mathbf{X}) \quad (0.60 > 0.40)$$

$\therefore$ Classify $\mathbf{X}$ as **Default** (C_2).

3.4 The Bayes Classifier Framework

3.4.1 Marginal Evidence Across K Classes

For a classification problem with K discrete classes $C_1, C_2, \dots, C_K$, the overall marginal probability of evidence $\mathbf{X}$ is:

$$P(\mathbf{X}) = \sum_{k=1}^K P(\mathbf{X} | C_k)P(C_k) \quad (3.11)$$

3.4.2 Optimal Bayes Decision Rule

The Bayes classifier assigns an observation $\mathbf{X}$ to the class maximizing the posterior probability:

$$\hat{c} = \arg \max_{C_k} P(C_k | \mathbf{X}) = \arg \max_{C_k} \frac{P(\mathbf{X} | C_k)P(C_k)}{P(\mathbf{X})} \quad (3.12)$$

Because the evidence denominator $P(\mathbf{X})$ is identical across all classes C_k , the decision rule simplifies to:

$$\hat{c} = \arg \max_{C_k} P(\mathbf{X} | C_k)P(C_k) \quad (3.13)$$

3.5 The Naive Bayes Classifier

3.5.1 The Challenge of High-Dimensional Likelihoods

When an observation is characterized by p features $\mathbf{X} = (x_1, x_2, \dots, x_p)$:

$$P(C_k | x_1, x_2, \dots, x_p) = \frac{P(x_1, x_2, \dots, x_p | C_k)P(C_k)}{P(x_1, x_2, \dots, x_p)} \quad (3.14)$$

Directly estimating the joint multivariate conditional distribution $P(x_1, x_2, \dots, x_p | C_k)$ from finite sample sizes is computationally and statistically challenging due to exponential data sparsity.

3.5.2 The Naive Conditional Independence Assumption

Definition 3.5.1: The Naive Assumption

All input features $x_1, x_2, \dots, x_p$ are mutually conditionally independent given the class label C_k :

$$P(x_1, x_2, \dots, x_p \mid C_k) = \prod_{j=1}^p P(x_j \mid C_k) \quad (3.15)$$

that is:

$$P(x_1, x_2, \dots, x_p \mid C_k) = P(x_1 \mid C_k) \cdot P(x_2 \mid C_k) \cdots P(x_p \mid C_k) \quad (3.16)$$

3.5.3 Naive Bayes Classification Rule

Applying the independence assumption produces the core operational classification formula:

$$\hat{c} = \arg \max_{C_k} P(C_k) \prod_{j=1}^p P(x_j \mid C_k) \quad (3.17)$$

Why Is It Termed “Naive”?

The assumption is termed “naive” because conditional independence among predictors is rarely strictly true in real-world scenarios.

Financial Example: In credit scoring, variables such as *Income*, *Debt*, and *Employment Status* clearly exhibit mutual dependencies. However, in practice, the naive approximation dramatically simplifies computation while delivering strong empirical classification performance.

3.5.4 Algorithmic Step-by-Step Sequence

Naive Bayes Execution Procedure

- i) **Start with Training Data:** Assemble feature vectors $\mathbf{X}_i$ and class labels y_i .
- ii) **Calculate Prior Probabilities:** Compute class base rates $P(C_k) = \frac{N_{C_k}}{N}$.
- iii) **Calculate Univariate Likelihoods:** Estimate 1-D conditional probabilities $P(x_j | C_k)$ for each feature given each class.
- iv) **Apply Naive Assumption:** Decompose joint likelihood into the product:

$$P(x_1, \dots, x_p | C_k) = \prod_{j=1}^p P(x_j | C_k)$$

- v) **Calculate Class Score:** Evaluate the unnormalized joint score for each candidate class:

$$\text{Score}(C_k) = P(C_k) \prod_{j=1}^p P(x_j | C_k)$$

- vi) **Compare Scores Across All Classes:** Evaluate $\text{Score}(C_k)$ across $k = 1, \dots, K$.
- vii) **Predict Output Class:**

$$\hat{c} = \arg \max_{C_k} \text{Score}(C_k)$$